SQL Questions on Derived Columns, Offset, Concat and Round

1. **Scenario:** Show each customer's full name and their ZIP in one readable column labeled `customer_label`.

```sql
SELECT CONCAT(customer_first_name, ' ', customer_last_name, ' – ZIP: ',
customer_zip) AS customer_label
FROM customer;
```

Explanation: Uses `CONCAT` to create a derived text column combining first name, last name and ZIP; `AS` names the derived column.

---

2. **Scenario:** For each `vendor_inventory` row, compute a `total_value` as `original_price * quantity` and show only rows where `total_value` > 50.

```sql
SELECT vendor_id, product_id, original_price, quantity,
       (original_price * quantity) AS total_value
FROM vendor_inventory
WHERE (original_price * quantity) > 50;
```

Explanation: A derived numeric column multiplies two columns inline; `WHERE` can reference the same expression.

---

3. **Scenario:** Display product name and a derived `label` that shows `name (size)` for easy reading.

```sql
SELECT product_name, CONCAT(product_name, ' (', product_size, ')') AS label
FROM product;
```

Explanation: `CONCAT` builds a human-friendly label using product attributes.

---

4. **Scenario:** Retrieve market dates with a derived `temp_range` (max - min) rounded to 1 decimal place.

```sql
SELECT market_date,
       ROUND((market_max_temp - market_min_temp), 1) AS temp_range
FROM market_date_info;
```

Explanation: Demonstrates numeric derived column and `ROUND(expr, 1)` to show one decimal.

---

5. **Scenario:** Show top 3 vendors by the summed `quantity` in `vendor_inventory` and include a rounded total.

```sql
SELECT vendor_id, ROUND(SUM(quantity), 2) AS total_quantity
FROM vendor_inventory
```

```
GROUP BY vendor_id
ORDER BY total_quantity DESC
LIMIT 3;
```

Explanation: Aggregation creates a derived value; `ROUND` sets precision; `LIMIT 3` returns top 3

---

6. **Scenario:** List 5 vendors but skip the first 2 alphabetically by `vendor_name` and show a label combining name and type.

```
SELECT CONCAT(vendor_name, ' – ', vendor_type) AS vendor_label
FROM vendor
ORDER BY vendor_name
LIMIT 2, 5;
```

Explanation: `ORDER BY` determines order, `LIMIT 2,5` skips two then returns five; `CONCAT` creates a display label.

---

7. **Scenario:** For each `customer_purchases` row show `customer_id`, `product_id`, and `cost` defined as `cost_to_customer_per_qty * quantity`, rounded to 2 decimals.

```
SELECT customer_id, product_id, ROUND(cost_to_customer_per_qty * quantity, 2)
AS cost
FROM customer_purchases;
```

Explanation: Runtime arithmetic forms a derived column; `ROUND(...,2)` formats monetary values.

---

8. **Scenario:** Create a `product_summary` that concatenates `product_name`, `size`, and `category_id`, then show only first 4 results.

```
SELECT CONCAT(product_name, ' | ', product_size, ' | cat:',
product_category_id) AS product_summary
FROM product
LIMIT 4;
```

Explanation: Multi-field `CONCAT` produces a compact summary; `LIMIT` restricts output for preview.

---

11. **Scenario:** Create `owner` full name for vendors, sort by last name, and show rows 4–

```sql
SELECT vendor_name, CONCAT(vendor_owner_first_name, ' ',
vendor_owner_last_name) AS owner
FROM vendor
ORDER BY vendor_owner_last_name
LIMIT 3, 5;
```

Explanation: Aliasing and `CONCAT` combine owner names; `LIMIT 3,5` provides offset-based paging.

---

16. **Scenario:** Compute `inventory_value` per vendor (sum of original_price * quantity), round to nearest integer, and display top 4 vendors.

```sql
SELECT vendor_id, ROUND(SUM(original_price * quantity), 0) AS inventory_value
FROM vendor_inventory
GROUP BY vendor_id
ORDER BY inventory_value DESC
LIMIT 0, 4;
```

Explanation: Aggregation produces a derived summary value; `ROUND(...,0)` gives integer.

---

17. **Scenario:** Create a `market_label` combining date and week (e.g., '2019-07-03 - Week 27') and show 3 results after skipping 2.

```sql
SELECT CONCAT(market_date, ' - Week ', market_week) AS market_label
FROM market_date_info
LIMIT 2, 3;
```

Explanation: Concise labeling of temporal data and offset paging.

---

18. **Scenario:** For `customer_purchases`, show `total_cost` and then filter rows where `total_cost` rounded to 1 decimal is equal to 10.0.

```sql
SELECT customer_id, product_id, ROUND(cost_to_customer_per_qty * quantity, 1)
AS total_cost
FROM customer_purchases
WHERE ROUND(cost_to_customer_per_qty * quantity, 1) = 10.0;
```

Explanation: Demonstrates using `ROUND` consistently in SELECT and WHERE to avoid floating-point mismatch.

---

19. **Scenario:** Display a `vendor_short` combining vendor id and the first 10 characters of vendor name; return rows 7–12.

```sql
SELECT vendor_id, CONCAT(vendor_id, ': ', SUBSTR(vendor_name,1,10)) AS
vendor_short
FROM vendor
LIMIT 6, 6;
```

Explanation: Shows how substring and concat create compact identifiers; `LIMIT 6,6` pages results.

---

20. **Scenario:** Show products with a derived `price_display` that appends currency symbol to `original_price` in `vendor_inventory` (string), rounded to 2 decimals.

```sql
SELECT product_id,
       CONCAT('$', ROUND(original_price, 2)) AS price_display
FROM vendor_inventory
LIMIT 0, 6;
```

Explanation: Combines numeric rounding with string concatenation to create formatted display fields.

---

25. **Scenario:** From `market_date_info` compute `avg_temp` as average of min and max temperatures, rounded to nearest integer, and present rows 2–6.

```sql
SELECT market_date, ROUND((market_min_temp + market_max_temp) / 2, 0) AS
avg_temp
FROM market_date_info
LIMIT 1, 5;
```

Explanation: Shows derived average calculation and paging via `LIMIT`.

---

26. **Scenario:** Produce a `compact_vendor` label that joins vendor name and vendor type, but only show those with type not equal to 'Eggs & Meats', offset first 3 and return 4.

```sql
SELECT CONCAT(vendor_name, ' - ', vendor_type) AS compact_vendor
FROM vendor
WHERE vendor_type <> 'Eggs & Meats'
LIMIT 3, 4;
```

Explanation: <> tests inequality; `CONCAT` creates the label; `LIMIT` demonstrates offset selection.

---

27. **Scenario:** For `vendor_inventory` create `rounded_qty` (quantity rounded to 1 decimal) and `unit_cost` (original_price/quantity rounded to 4 decimals); return first 10.

```sql
SELECT vendor_id, product_id,
       ROUND(quantity, 1) AS rounded_qty,
       ROUND(original_price / NULLIF(quantity,0), 4) AS unit_cost
FROM vendor_inventory
LIMIT 0, 10;
```

Explanation: Multiple derived columns with different rounding precisions; `NULLIF` avoids division by zero.

---

28. **Scenario:** Show a `label` for each booth that combines booth number and price level and then show records 10–15 (if present).

```sql
SELECT CONCAT('Booth#', booth_number, ' [', booth_price_level, ']') AS label
FROM booth
LIMIT 9, 6;
```

Explanation: Composes an informative label and demonstrates mid-table paging.

---

29. **Scenario:** Create `purchase_value` per `customer_purchases` and then fetch rows where the rounded value equals 20.00.

```sql
SELECT customer_id, product_id, ROUND(cost_to_customer_per_qty * quantity, 2)
AS purchase_value
FROM customer_purchases
WHERE ROUND(cost_to_customer_per_qty * quantity, 2) = 20.00;
```

Explanation: Illustrates equality checks on rounded monetary derived columns.

---

32. **Scenario:** Display a `date_and_week` column formatted via concatenation and return rows 3–7.

```sql
SELECT CONCAT(market_date, ' | W', market_week) AS date_and_week
FROM market_date_info
LIMIT 2, 5;
```

Explanation: Simple derived text combining date and numeric week; `LIMIT` shows paging.

---

33. **Scenario:** For `vendor_inventory`, make a `markup_price` by adding 20% to original_price and round to 2 decimals. Show the second page (rows 11–15).

```sql
SELECT vendor_id, product_id, ROUND(original_price * 1.2, 2) AS markup_price
FROM vendor_inventory
LIMIT 10, 5;
```

Explanation: Derived pricing formulas and paging to examine later rows.

37. **Scenario:** Create `qty_bucket` as rounded quantity to nearest integer and show vendors whose bucket >= 10, returning rows 0–5.

```
SELECT vendor_id, ROUND(quantity, 0) AS qty_bucket
FROM vendor_inventory
WHERE ROUND(quantity, 0) >= 10
LIMIT 0, 6;
```

Explanation: Rounding used to bucket continuous values for filtering.

---

39. **Scenario:** From `customer_purchases`, compute `unit_cost` showing rounded value and then fetch rows 20–25 (simulate deeper paging).

```
SELECT customer_id, product_id, ROUND(cost_to_customer_per_qty, 3) AS
unit_cost
FROM customer_purchases
LIMIT 19, 6;
```

Explanation: Presents unit-level pricing rounded to 3 decimals and demonstrates jumping to later records.

---

40. **Scenario:** Create a `market_summary` string that shows date and a rounded `avg_temp`, then show the first 10.

```
SELECT CONCAT(market_date, ' – AvgTemp: ', ROUND((market_min_temp +
market_max_temp)/2,1)) AS market_summary
FROM market_date_info
LIMIT 0, 10;
```

Explanation: Combined derived text with rounded numeric calculation for clear reporting.

---

43. **Scenario:** Create `name_zip` combining customer last name and ZIP in parentheses and show rows 2–6.

```
SELECT CONCAT(customer_last_name, ' (', customer_zip, ')') AS name_zip
FROM customer
LIMIT 1, 5;
```

Explanation: Simple derived label combining identity and location; offset shows subset.

---

45. **Scenario:** Produce `compact_purchase` combining customer id, product id and rounded quantity, then return rows 5–10.

```
SELECT CONCAT('C', customer_id, '-P', product_id, ':', ROUND(quantity,2)) AS
compact_purchase
FROM customer_purchases
LIMIT 4, 6;
```

Explanation: Compact derived strings useful in logs; offset selects a specific slice.

---

49. **Scenario:** For `customer` table produce a `contact_card` combining name and ZIP, and then show customers after skipping 10 returning next 5.

```
SELECT CONCAT(customer_first_name, ' ', customer_last_name, ' — ZIP:',
customer_zip) AS contact_card
FROM customer
LIMIT 10, 5;
```

Explanation: Larger offset demonstrates navigation through a long customer list.
